Egg Segmentation: SAM vs. Classical Image Processing, and Egg Tracking

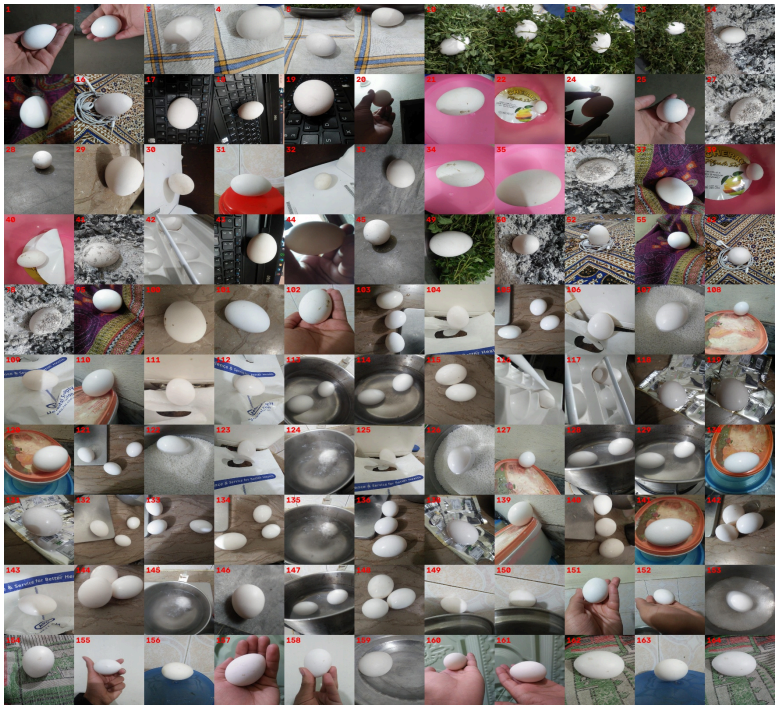
CV Intern Task · Duggineni Akhil · Code: github.com/Duggineniakhil/Egg_Segmentation · Tracking video: **(Google Drive link to be added)**

Summary

- **Task 1:** pseudo ground-truth masks for all 110 images from **Grounding DINO + SAM ViT-H**: 213 eggs, with 3 images fixed by hand-written prompts.
- **Task 2:** a **scene-grouped 4:1 split** (87 dev / 23 eval images, 4 whole eval scenes) that measurably reduces leakage, and a fully classical segmenter developed over **30 logged trials**. The key steps were a multi-channel threshold sweep, an egg-shaped candidate score, a nested-candidate upgrade and GrabCut. Dev mIoU went from 0.339 to **0.786**.
- **Task 3:** on held-out eval, **mIoU 0.789 (95% CI 0.68–0.85), Dice 0.855, instance F1 0.929, 100% egg-count accuracy**. The main error is under-segmentation along an egg's lit/shadow line (38% of lost IoU). Also: an ablation, scene-level confidence intervals, and a follow-up experiment (v2) that fixes that error on dev but not on eval, reported as such.
- **Task 4:** the classical detector plus a tracker that follows eggs **through hand pick-ups** with optical flow. Against SAM ground truth on 196 frames: **MOTA 0.962, IDF1 0.953, 0 ID switches**. An earlier version swapped the two eggs at 30 s (IDF1 0.62); the cause and the fix are described.

Dataset

The Roboflow "egg" v1 dataset has **110 images (640×640)** of 1–4 white eggs on very different backgrounds: hands, keyboards, leaves, ash, pink bowls, steel pots, white bags and egg trays. The export only sorts images into classification folders, so there are **no box or mask labels**. Task 1 has to produce the masks from scratch.



All 110 images. The same physical scene appears under non-consecutive file numbers (e.g. the steel pot: 113, 114, 124, 128, 129, 135, 145, 147, 153, 159).

Task 1: Segmentation with a pretrained model

Pipeline (`notebooks/task1_sam_kaggle.ipynb`, Kaggle T4 GPU):

1. **Boxes:** Grounding DINO (base), an open-vocabulary detector, finds eggs with the text prompt `"an egg."` (box threshold 0.35, text threshold 0.25).
2. **Box cleanup:**
   - NMS (IoU 0.5).
   - Boxes under 0.2% of the image are dropped.
   - "Group" boxes are dropped: boxes containing ≥ 2 other boxes, which Grounding DINO often adds around a cluster of eggs.
3. **Masks:** SAM ViT-H is prompted with each box.
4. **Mask cleanup:**
   - Pieces smaller than 15% of the largest piece are dropped. The large pieces are all kept, because an occluder can split one egg.
   - Holes are filled.
   - Where masks overlap, a pixel goes to the mask with the higher SAM IoU prediction.
5. **Output:** an instance label map per image (`outputs/sam_masks/*.png`; 0 = background, 1…N = eggs).

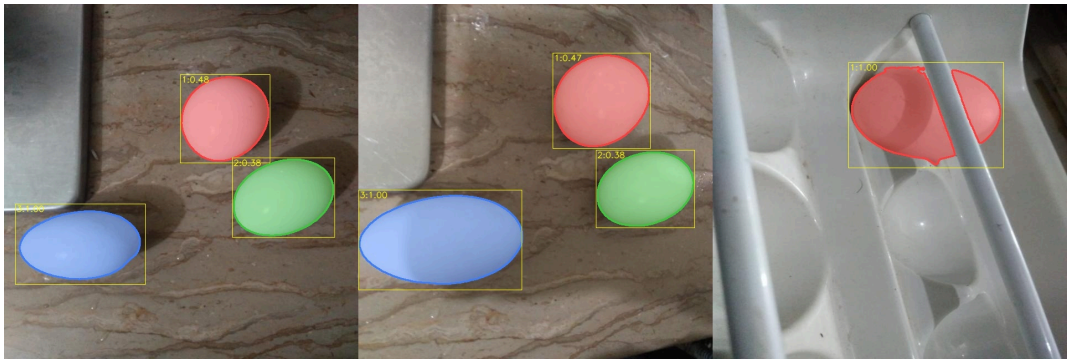
Review and fixes. I checked every image on review sheets. 107 of 110 were correct on the first run. Three needed hand-written prompts, recorded as `manual` in the meta files:

- **105 and 134:** an egg in shadow was missed (its box scored just under the threshold). Lowering the threshold everywhere would have added false boxes elsewhere, so I added one box each.
- **117:** an egg in a tray cup is split by a rod. SAM picked the cup rim, so I added a box plus one positive click on each side of the rod.

The review also exposed a bug: the first mask cleanup kept only the largest piece, so the far side of an occluded egg was dropped. Hence the "≥ 15% of the largest piece" rule above. The final result is **213 eggs** (93 images with one egg, 6 with two, 11 with three).



Part of the SAM review sheet (multi-egg images): boxes with detection score and instance masks.



The three hand-fixed images: 105 and 134 (missed egg added), 117 (egg split by the tray rod).

Limitation: these masks are *pseudo* ground truth. In #123 and #125, SAM excludes the part of the egg hidden behind paper, and #117 has a small spike. All later "errors" are disagreement with SAM, not with a human.

Task 2: Segmentation with classical image processing

2.1 Leakage-aware dev/eval split

The leakage problem. Leakage here comes from **scenes**, not augmentation (Roboflow applied none). The images are photos of about two dozen physical setups taken from slightly different angles. A random split would put near-copies of a scene in both dev and eval, so eval would mostly measure whether the method remembers a background.

Method (`src/make_split.py`):

1. **Inpaint the eggs out** using the SAM masks. All eggs look alike, so they would pull unrelated scenes together; the background is what identifies a scene.
2. **Embed** each background with CLIP ViT-B/32.
3. **Cluster** with agglomerative clustering (cosine distance 0.15).
4. **Review by eye.** The clustering split some real scenes apart.
   - A coarser threshold only merged them after unrelated scenes had collapsed into a 47-image group.
   - Adding a colour histogram fixed one scene and broke others.

   So **9 scenes confirmed by eye were merged by hand** (`SCENE_OVERRIDES`, each commented).
5. **Assign whole scenes to eval:** a random search over scene subsets for about 20% of images and a similar share of multi-egg images.

	dev	eval
images	87	23
scenes	19	4 (steel pot, orange tub, striped cloth, green door + hand)
multi-egg images	13.8%	21.7%

The split is 3.8:1 rather than exactly 4:1, because whole scenes can't be divided.

Leakage check: an eval image's mean highest CLIP similarity to any dev image is **0.859**, against **0.911** for random image-level splits. Section 3.6 shows why this mattered.

2.2 Final algorithm (v1, `configs/t16_nest_cap4.yaml`)

All code is in `src/classical_seg.py`, with every stage set from a YAML config.

1. **Gaussian blur** (7×7).
2. **Threshold sweep instead of one threshold.** Thresholds are taken at 24 percentile levels on three channels:
   - **gray**, in both polarities;
   - **smoothness**: 255 − 8 × the local standard deviation in an 11×11 window. Eggs are smooth, while ash, leaves, cloth patterns and keyboards are textured.
   - **Lab whiteness**: lightness L − 2 × chroma.

   Each binary mask is **cut along Canny edges** (auto thresholds, σ 0.6), so an egg separates from a neighbour of similar brightness. Every connected blob is a candidate.
3. **Candidate filter:** egg-sized (1.5–50% of the image), solid (solidity ≥ 0.85), aspect ratio ≤ 2.5 and ellipse-like (IoU with its fitted ellipse ≥ 0.75).
4. **Candidate score:**

   $$\text{quality} = \text{shape}^2 \times \min(1, \text{contrast} / 50) \times 0.6 \text{ if touching the image border}$$

   - *shape* rescales the ellipse IoU from [0.75, 0.97] to [0, 1].
   - *contrast* is the egg's mean gray minus a 12 px ring around it.
5. **Greedy non-overlapping selection,** at least 1 and at most 6 eggs. Each pick is **upgraded to the largest candidate that contains it**, if that candidate's quality is ≥ 0.3× the pick's and it is ≤ 4× larger. This replaces a

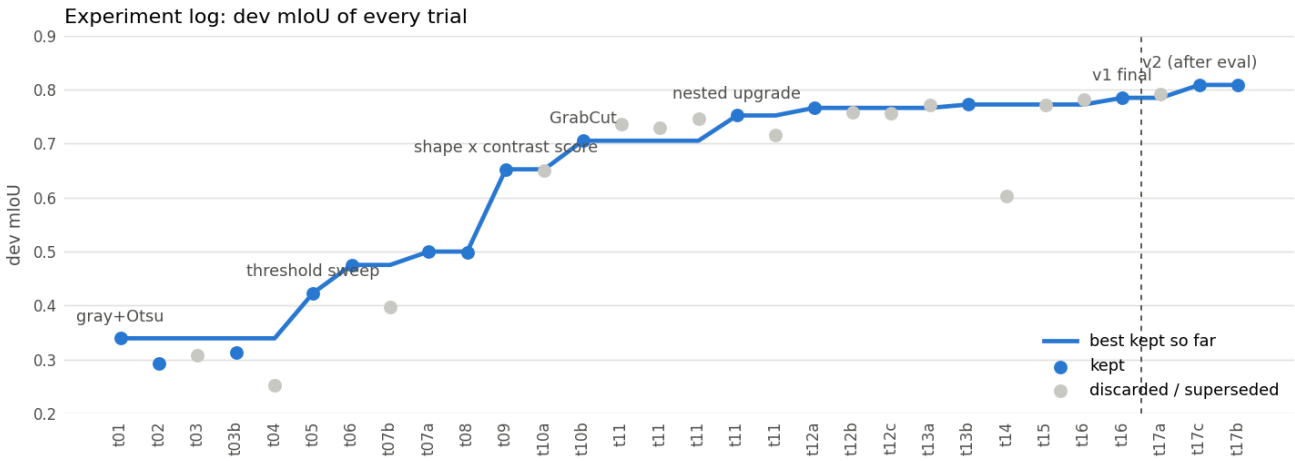
bright cap with the whole egg.

6. **GrabCut refinement:** the eroded blob is sure foreground and a 30 px band around it is "unsure". Its random initialisation is seeded per image, so results don't depend on processing order.

On one CPU thread this takes 4.4 s per image (GrabCut 45%). SAM took 2.6 s per image on a T4 GPU.

2.3 Experiment log: how the trials led to the algorithm

Each trial is a config inheriting from its predecessor (`base:`), and states what changed and why. It was run on **dev only** with `src/run_experiment.py`, which appends to `outputs/experiments/log.csv`. The full log is on the next pages, including every discarded attempt. The main path was:



- **t01–t04: the limit of single thresholds.** Gray + Otsu gives 0.339 mIoU, finding eggs plus every bright patch (precision 0.37). Cleanup, whiteness and shape filters didn't help (0.25–0.31). On white or bright backgrounds the egg *merges* with the background into one blob, and a backlit egg is darker than its surroundings. **No single threshold separates them.** The shape filter raised instance precision to 0.81 while recall collapsed: what survives is an egg, but most eggs never become a blob.
- **t05: threshold sweep, both polarities** (0.423). Some threshold isolates the egg in many more images.
- **t06–t07: cutting the masks along Canny edges** separates eggs from similar neighbours (0.500). Loosening the shape filter (t07b) **hurt**: more junk candidates beat the eggs.
- **Diagnosis tools that shaped the next steps:**
  - `src/diagnose_sweep.py` measures the **oracle IoU**, the best IoU any candidate reaches. It shows whether a failure is in *generation* (no good candidate exists) or in *selection*. After t05, 42 of 50 failures were generation failures. After **t08 (smoothness channel)** the oracle rose from 0.67 to 0.78, but the final mIoU didn't move: **selection had become the bottleneck**.
  - `src/analyze_candidates.py` measures, over all dev candidates, how well each feature separates eggs (IoU ≥ 0.8 with SAM) from non-eggs (IoU < 0.3), as an AUC:

Feature	AUC
ellipse IoU	0.976
ring contrast	0.941
Canny edge support (the old score's main term)	only 0.702
old score	0.760
shape² × contrast × border penalty	**0.988**

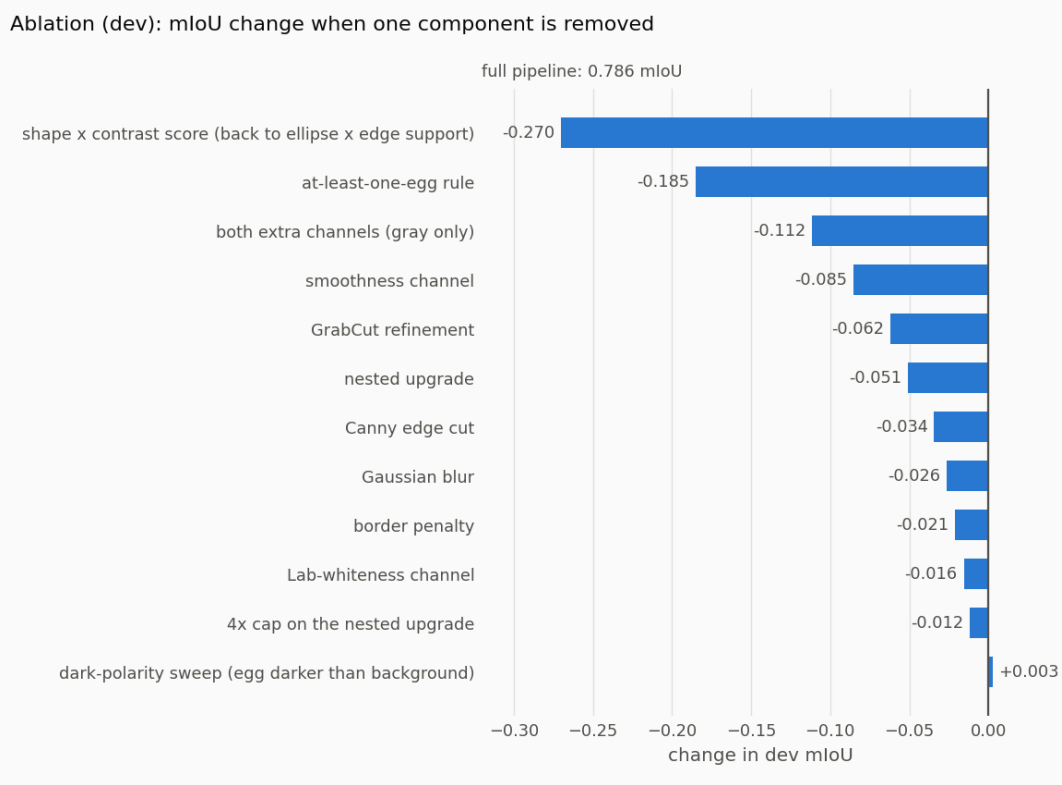
- **t09: the new score** (0.652, count accuracy 0.98) was the biggest single step.
- **t10b: GrabCut** (0.705, boundary F1 0.43 → 0.62). Replacing blobs by fitted ellipses (t10a) didn't help.
- **t11: nested upgrade.** A loss breakdown showed that 51% of lost IoU was *partial* eggs (precision 0.96, recall 0.61): a bright cap fits an ellipse slightly better than the whole egg. Upgrading to the containing candidate, with the

ratio swept over 0.85, 0.7, 0.5, 0.3 and 0.15, peaked at 0.3 (0.752).

- **t12–t13:** a wider GrabCut band (0.766) and a Lab-whiteness channel (0.772). Two GrabCut growth limits were discarded, because they held back more partial eggs than the leaks they stopped.
- **t14–t16: three failed fixes that each explained something.**
  - **Not forcing an egg (t14):** −0.17, because many true eggs score low.
  - **Undoing implausible GrabCut growth (t15):** no effect, because the leaks happen *before* GrabCut.
  - **That led to t16:** capping the nested upgrade at 4× growth (0.785; #32: 0.08 → 0.94). This is the **frozen v1** model.

2.4 Ablation (dev, `src/ablation.py`)

Removing one component at a time from v1 confirms that the parts the log added are the ones that matter. The **candidate score** matters most (−0.270), then the at-least-one-egg rule (−0.185) and the smoothness channel (−0.085). GrabCut is worth 0.062 mIoU but 0.237 boundary F1. **The dark-polarity sweep adds nothing** (+0.003 when removed; it was made redundant by the smoothness channel). It stays in the frozen model, because removing it after evaluation would be tuning on eval information.



Trial	Change (what and why)	dev mIoU	Decision
`t01_gray_otsu`	Baseline: grayscale + global Otsu threshold, every connected component is an egg. *Why:* Eggs are the brightest objects in most images; a simplest-possible reference point	0.339	reference
`t02_cleanup`	Gaussian blur 7, open 5 / close 15, fill holes, keep blobs 1.5%-50% of the image. *Why:* t01 output is thousands of specks; dev eggs cover 2.6%-38% of the image (SAM masks), so size alone removes most noise	0.293	keep (needed for shape filters; mIoU drop exposes egg-background merging)
`t03_lab_whiteness`	Score = Lab L - 2*chroma instead of grayscale. *Why:* Eggs are white (low chroma); pink bowls, leaves, orange tubs and purple cloth are bright but colourful	0.308	discard (t03b better)
`t03b_hsv_whiteness`	Score = V*(1-S) in HSV (alternative whiteness measure to t03). *Why:* Compare with the Lab version; HSV saturation is unstable for dark pixels	0.314	keep
`t04_shape_filter`	Reject blobs with solidity < 0.85 or ellipse-fit IoU < 0.75. *Why:* Dev eggs: solidity p5 0.97, ellipse IoU p5 0.89 (min 0.66 when occluded); background blobs are irregular	0.251	discard (inst precision up, recall collapses: single threshold is the limit)
`t05_sweep`	Replace the single threshold with a 24-level threshold sweep over the gray image, both polarities; candidates scored by ellipse IoU x Canny edge support, greedy non-overlapping pick (min 1, max 6 eggs). *Why:* t02-t04 show one global threshold can't isolate eggs on white or bright backgrounds (egg merges with them) or when backlit (egg darker); some threshold isolates the egg in almost every image	0.423	keep (+0.11 mIoU over t03b)
`t06_edge_cut`	Cut every sweep mask along dilated Canny edges before finding blobs (rim given back afterwards). *Why:* diagnose_sweep on t05: 42/50 failures are generation failures; the egg stays connected to a neighbouring region of similar brightness at every threshold, but there is a visible edge between them	0.475	keep
`t07b_relax_shape`	min_solidity 0.85 -> 0.8, min_ellipse_iou 0.75 -> 0.65, max_area_frac 0.5 -> 0.6. *Why:* several t06 generation failures (ash, occluded, large close-up eggs) are just outside the shape/size limits; dev GT minimum ellipse IoU is 0.66	0.397	discard (more junk wins selection)
`t07a_canny_wide`	Canny sigma 0.33 -> 0.6 (lower low-threshold, more edges). *Why:* white-on-white outlines (bag, tray, paper) are faint; auto-Canny around the median misses them, so the egg still leaks into the background	0.500	keep
`t08_smoothness`	Add a second sweep channel: smoothness = 255 - 8*local std (11x11); candidates from both channels compete. *Why:* eggs are smooth while ash, leaves, patterned cloth and keyboards are textured; gives the sweep an independent chance where brightness alone merges egg and background	0.498	keep (oracle IoU 0.67->0.78; selection now the bottleneck)
`t09_shape_contrast`	Candidate quality = shape^2 * min(1, contrast/50) * (0.6 if touching image border), replacing ellipse_iou * edge_support; min_quality 0.35 -> 0.3. *Why:* after t08 the egg is usually among the candidates but loses at selection (17 failures). On dev candidates, ellipse IoU (AUC 0.976) and ring contrast (0.941) separate eggs far better than edge support (0.702); the combination gets AUC 0.988 vs 0.760	0.652	keep (+0.15 mIoU)
`t10a_ellipse`	Replace each selected blob by its fitted ellipse. *Why:* t09 precision 0.91 but recall 0.68: selected blobs are usually a correct but partial	0.650	discard (no gain)

Trial	Change (what and why)	dev mIoU	Decision
	egg (shadowed side cut off); eggs are near-ellipses (dev GT ellipse IoU median 0.976), so the fit should restore the missing part		
`t10b_grabcut`	GrabCut refinement: eroded blob = sure egg, 15 px band = unsure, 5 iterations. *Why:* same under-segmentation as t10a, but let colour models decide the boundary instead of assuming a perfect ellipse (occluded eggs are not ellipses)	0.705	keep
`t11_nested_07`	After picking a blob, upgrade it to the largest candidate containing >=90% of it whose quality is >= 0.7 x the pick's. *Why:* t10b loss breakdown: 51% of lost IoU is partial eggs, 29/31 under-segmented (precision 0.96, recall 0.61); the sweep makes nested blobs (bright cap inside whole egg) and the cap often wins on ellipse fit	0.736	superseded by ratio sweep
`t11_nested_085`	After picking a blob, upgrade it to the largest candidate containing >=90% of it whose quality is >= 0.85 x the pick's. *Why:* t10b loss breakdown: 51% of lost IoU is partial eggs, 29/31 under-segmented (precision 0.96, recall 0.61); the sweep makes nested blobs (bright cap inside whole egg) and the cap often wins on ellipse fit	0.729	superseded by ratio sweep
`t11_nested_05`	Nested upgrade with quality ratio 0.5 (looser than 0.7). *Why:* 0.7 beat 0.85 (+0.031 vs +0.024 mIoU); check whether a looser ratio recovers more partial eggs	0.745	superseded by ratio sweep
`t11_nested_03`	Nested upgrade with quality ratio 0.3 (looser than 0.7). *Why:* 0.7 beat 0.85 (+0.031 vs +0.024 mIoU); check whether a looser ratio recovers more partial eggs	0.752	keep (peak of ratio sweep, +0.047 mIoU)
`t11_nested_015`	Nested upgrade with quality ratio 0.15. *Why:* mIoU still rising at 0.3 while precision falls; find where the curve turns	0.716	discard (upgrades to oversized blobs, precision 0.83)
`t12a_band30`	GrabCut unsure band 15 -> 30 px. *Why:* remaining partial eggs: GrabCut cannot grow past the 15 px band, so a blob covering half an egg stays half	0.766	keep (+0.014)
`t12b_band30_limit`	GrabCut band 30 px, growth limited to the blob's fitted ellipse x1.3. *Why:* keep the extra reach of t12a but stop leaks into skin/walls of similar colour (#20, #155)	0.759	discard (limit blocks more partial-egg growth than leaks it stops)
`t12c_band45_limit`	GrabCut band 45 px, growth limited to fitted ellipse x1.5. *Why:* check whether even more reach helps once growth is bounded	0.755	discard
`t13a_ch_hsv`	Third sweep channel: HSV whiteness V*(1-S). *Why:* 12 dev images still have the wrong/missing egg (42% of lost IoU), mostly because the egg never forms a clean blob; whiteness separates white eggs from coloured surroundings (pink bowl, hand) that gray and smoothness merge	0.771	discard (t13b marginally better)
`t13b_ch_lab`	Third sweep channel: Lab whiteness L - 2*chroma. *Why:* same as t13a with the Lab whiteness measure	0.772	keep (marginal +0.006)
`t14_no_forced_egg`	min_eggs 1 -> 0: output nothing when no candidate reaches min_quality. *Why:* in the missed-egg images the forced pick is a background blob (#11, #106, #117, #125); it scores IoU 0 either way, but adds a false-positive egg	0.603	discard (-0.17 mIoU: many true eggs score below min_quality; every image has >=1 egg)
`t15_revert_leaks`	After GrabCut, per egg: revert to the pre-GrabCut blob if the area grew >2x or the result's ellipse IoU < 0.8. *Why:* t13b worst cases #32, #20, #155: egg correctly selected, then GrabCut floods white paper / skin / wall; the ellipse growth limit (t12b) was too blunt	0.771	discard (no effect: the leaks happen before GrabCut, in the nested upgrade)

Trial	Change (what and why)	dev mIoU	Decision
	and hurt partial eggs overall, so only undo GrabCut where its output is implausible		
`t16_nest_cap25`	Nested upgrade may grow a pick at most 2.5x in area. *Why:* t15 found the #32/#20/#155 leaks happen before GrabCut: the nested upgrade (ratio 0.3) swaps a correct egg for a 4-11x larger egg+paper / egg+hand region that contains it	0.782	superseded by 4x
`t16_nest_cap4`	Nested upgrade may grow a pick at most 4x in area. *Why:* t15 found the #32/#20/#155 leaks happen before GrabCut: the nested upgrade (ratio 0.3) swaps a correct egg for a 4-11x larger egg+paper / egg+hand region that contains it	0.785	keep, FINAL v1 (frozen, evaluated on eval)
`t17a_complete_canny`	(v2, after the v1 eval run) Ellipse completion before GrabCut: fit an ellipse to the outline points of each selected blob that lie on a Canny edge, add it to the blob (growth <= 2.5x). *Why:* 38% of lost IoU is eggs split along the lit/shadow line. Diagnosis on dev: only 4% of the cut's points lie on a Canny edge vs 25% of the real outline, so an ellipse through the edge-supported points should recover the shadowed half	0.791	discard (+0.005: fires on only 24/87 images)
`t17b_complete_relative`	Ellipse completion with relative edges: outline points whose gradient is >= 0.3 x the 90th percentile along that blob's own outline. *Why:* t17a fired on only 24/87 images: 20 of 33 under-segmented eggs have <20 outline points on any Canny edge (the whole outline is faint on pale backgrounds); within one blob the real outline is still stronger than the soft cut	0.808	keep (+0.022 mIoU, +0.031 boundary F1)
`t17c_complete_chroma`	Completed pixels must be about as colourless as the egg (chroma <= max(12, 1.5 x the blob's 90th-percentile chroma)). *Why:* t17b's 4 worse images: the ellipse overshoots onto leaves (#10), a patterned rug (#162), a pink bowl (#22), a pale wall (#158); a shadowed egg side is still neutral, most overshoots are not	0.809	keep, FINAL v2 (+0.001, fewer images made worse: 3 vs 4)

Full experiment log (dev split). Trials t17a–c were run after the v1 evaluation (see 3.6).

Task 3: Evaluation and analysis

The Task 1 SAM masks are the ground truth and the classical v1 outputs are the predictions (`src/evaluate.py`). **v1 was frozen and run on eval once.**

3.1 Quantitative results

Metric	dev (87 images, 110 eggs)	**eval (23 images, 28 eggs)**
mean IoU	0.786	**0.789**
mean Dice	0.837	**0.855**
pixel precision / recall	0.895 / 0.828	**0.905 / 0.830**
boundary F1 (3 px)	0.699	**0.632**
HD95, median (px)	16.8	**24.0**
instance F1 (Hungarian, IoU ≥ 0.5)	0.885	**0.929**
correct egg count	97.7%	**100%**

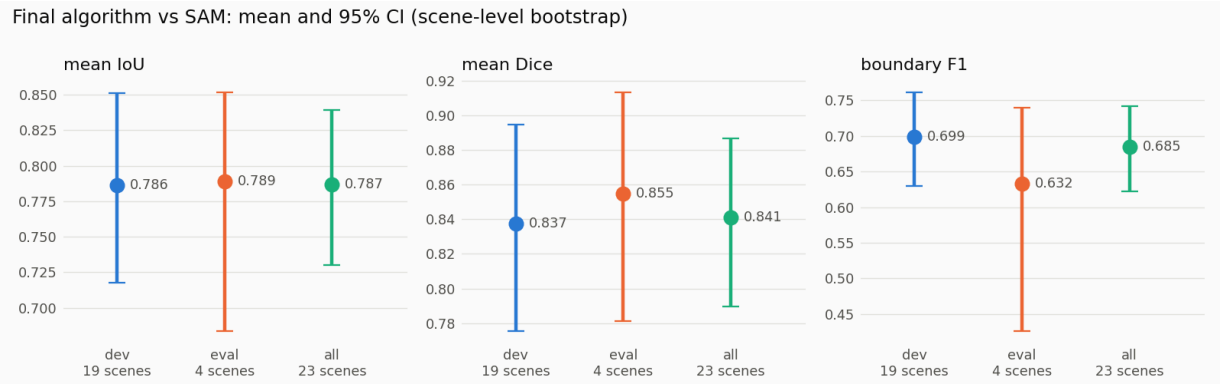
Why these metrics:

- **IoU and Dice** measure overall agreement.
- **Precision and recall** tell *which way* a mask is wrong.
- **Boundary F1 and HD95** measure contour quality, which region overlap hides.
- **Instance F1 and count accuracy** check that eggs come out as separate objects, which Task 4 needs.

Uncertainty (`src/stats.py`). Images of one scene are correlated, so whole scenes are resampled (cluster bootstrap):

	mIoU, 95% CI
eval	[0.683, 0.852]
all 110 images	[0.730, 0.839]

Over 5 scene-grouped folds of all images, mIoU is 0.787 ± 0.070 (0.72–0.90). Dev and eval agree. **Which scenes are tested moves the result more than each of the late tuning steps.**

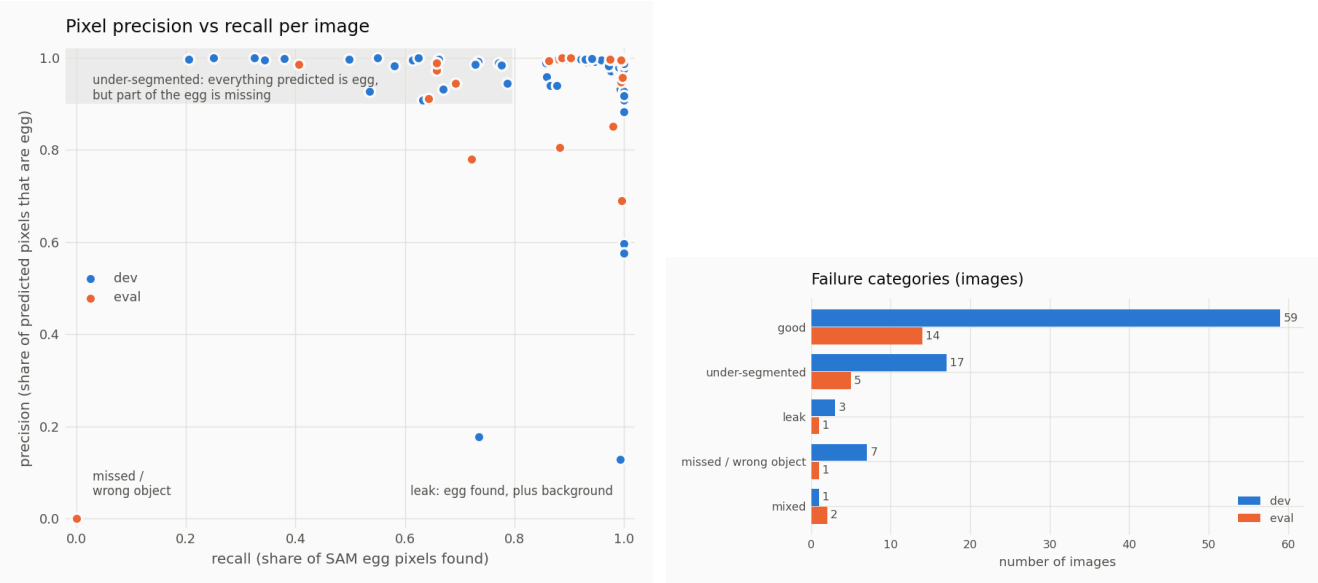


3.2 Where the error comes from

Each image is put into a category from its pixel precision P and recall R:

- **good:** IoU ≥ 0.8.
- **under-segmented:** P ≥ 0.9 and R < 0.8.
- **leak:** P < 0.75 and R ≥ 0.75.
- **missed / wrong object:** R < 0.3.
- **mixed:** everything else.

Category	images	share of lost IoU
good	73	15%
under-segmented	22	**38%**
missed / wrong object	8	32%
leak	4	9%
mixed	3	6%



Left: most imperfect images sit top-left. Everything predicted is egg, but part of the egg is missing. Right: categories per split.

3.3 What makes an image hard

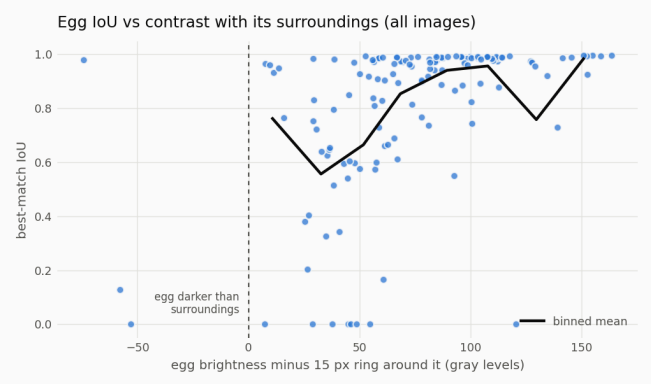
- **Contrast with the surroundings is the main driver** (Spearman ρ = 0.59, p = $4 \cdot 10^{-14}$):

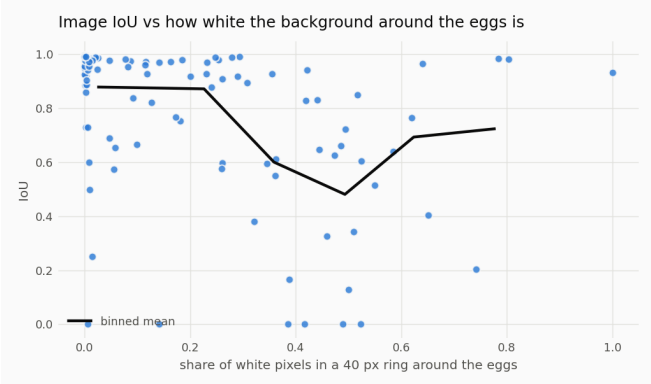
Contrast (egg vs 15 px ring)	eggs	mean IoU
≥ 50 gray levels	102	0.89
0–50	33	0.56
< 0 (backlit)	3	0.37

- **Partly white backgrounds are hardest:**

Background (white share of a 40 px ring)	images	mean IoU
< 20% white	66	0.88
20–60%	37	0.63
> 60%	7	0.75

Fully white surroundings do a bit better, because the egg's shadow then gives a clean edge. - **Touching eggs** score 0.68 vs 0.82. **Egg size has no clear effect.** - **By scene,** the worst are eggs held against a bright wall (0.37), a red lid / steel tray (0.49), the white egg tray (0.54) and the "Better Health" bag (0.58).

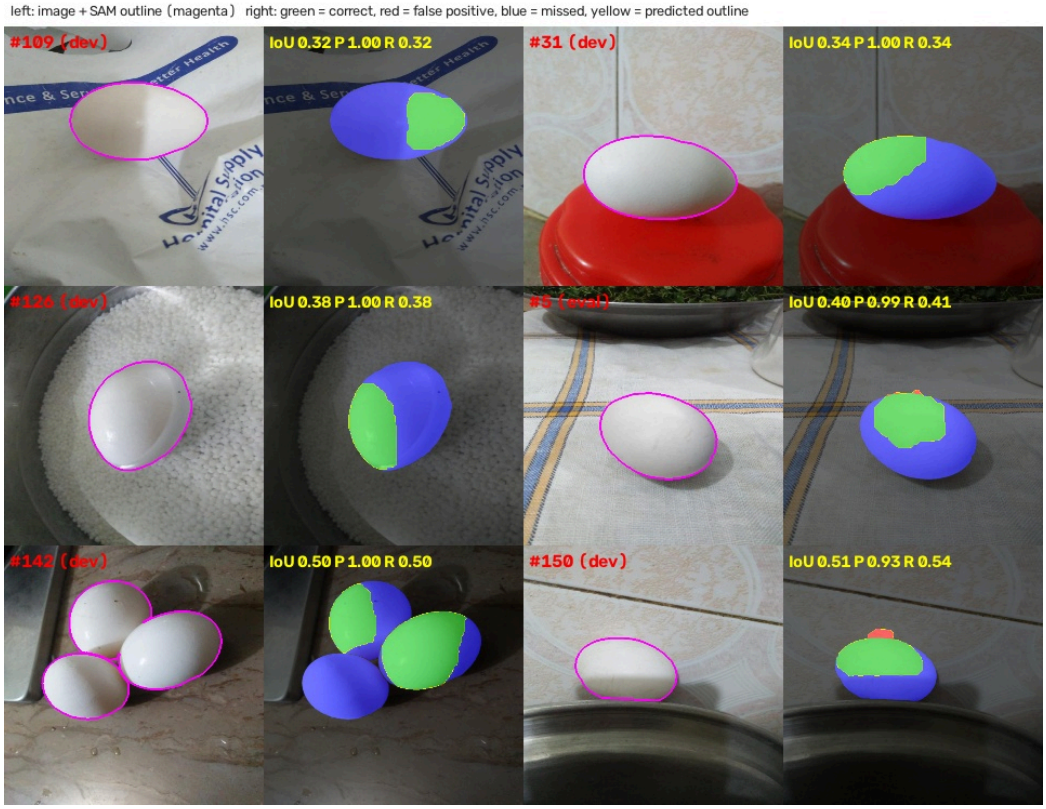




3.4 Qualitative analysis: what went wrong

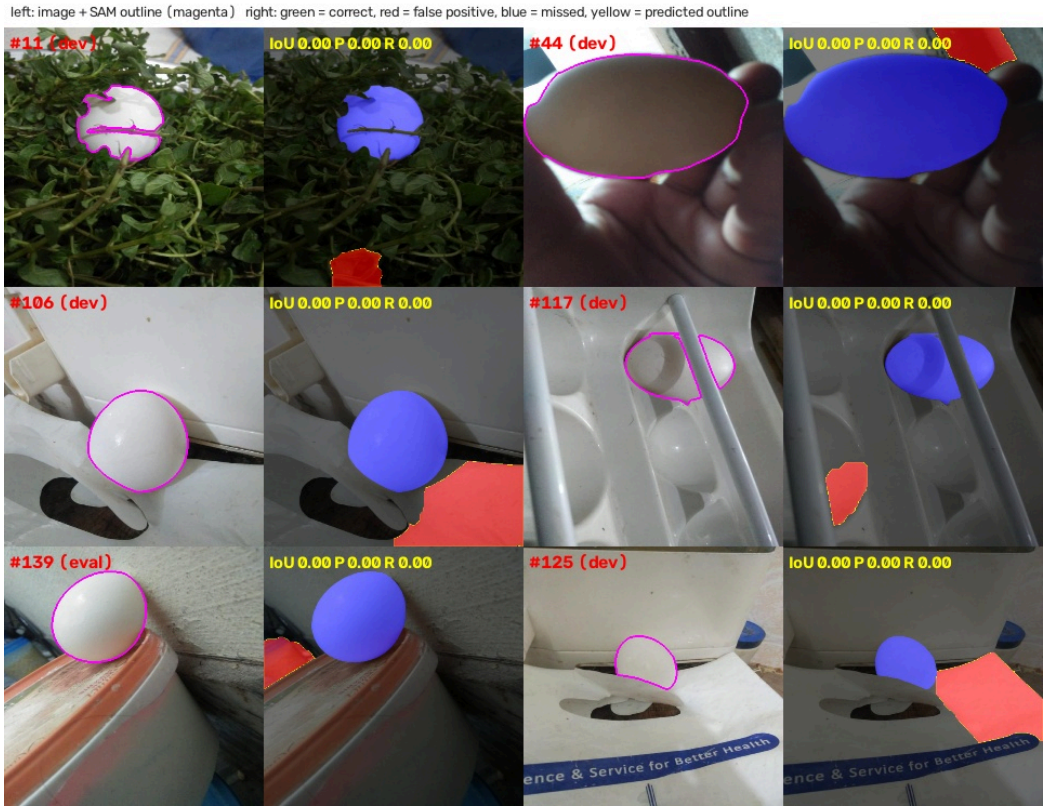
In each gallery, the left of every pair is the image with the SAM outline (magenta). The right is the error map: green = correct, red = false positive, blue = missed.

Under-segmented. The prediction stops at the **line between the lit and shadowed sides of the egg**. Under one-sided light a white egg has two very different brightness levels. At every sweep threshold the two halves are separate blobs, and the lit half alone is already a good ellipse, so it wins. GrabCut then can't cross into the shadowed half, because its colour model learned "egg = bright".



Missed / wrong object. The causes:

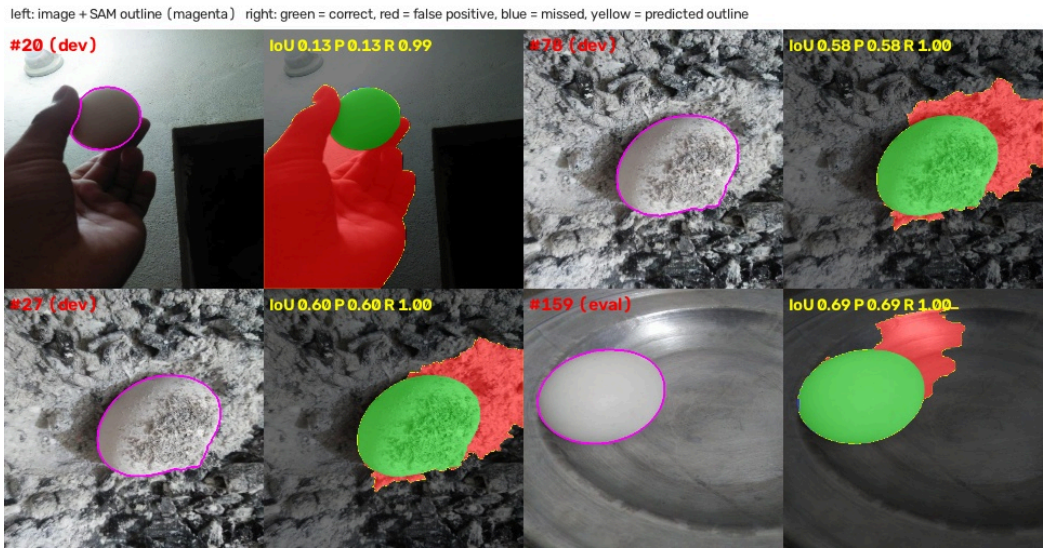
- **White egg on a white bag** (#106, #125): no threshold isolates the egg, and the forced single egg lands on a patch of bag.
- **Occlusion** (#11, #117): a twig or rod cuts the egg into pieces that aren't egg-shaped on their own.
- **A close-up held egg** (#44) that merges with the bright background.
- **An egg cut by the frame edge** (#139).



Leak. The egg is found, but the mask takes in some background:

- **#20:** a backlit egg plus the hand holding it.
- **#27, #78:** ash-covered eggs, where GrabCut spreads into the ash heap.
- **#159:** reflections on steel.

Backlighting alone isn't fatal: #24 is backlit and scores 0.98.



3.5 Performance gaps and potential improvements

Gap	Evidence	Improvement
Lit/shadow split	38% of lost IoU	Ellipse completion from edge-supported outline points (tried in 3.6), or shading-invariant chromaticity
Low contrast, white on white	IoU 0.56 at contrast < 50	Use the egg's own shadow as a cue, or a gradient-based ellipse detector (randomized Hough) independent of region thresholds
Occlusion	#11, #117	RANSAC ellipse over several nearby egg-coloured pieces
Forced pick lands on background	#106, #125, #139	Keep the rule (t14: −0.17 without it), but choose the forced pick with a stricter shape test
Leaks into skin, ash, reflections	#20, #27, #78, #159	Skin-colour model as GrabCut background seeds; a specular-reflection test
Boundary accuracy	boundary F1 0.63–0.70	Active contours fitted to the image gradient

3.6 A follow-up after the frozen evaluation: ellipse completion (v2)

Diagnosis (dev only). On the lit/shadow cut, only **4%** of the outline points lie on a Canny edge, against 25% of the real outline.

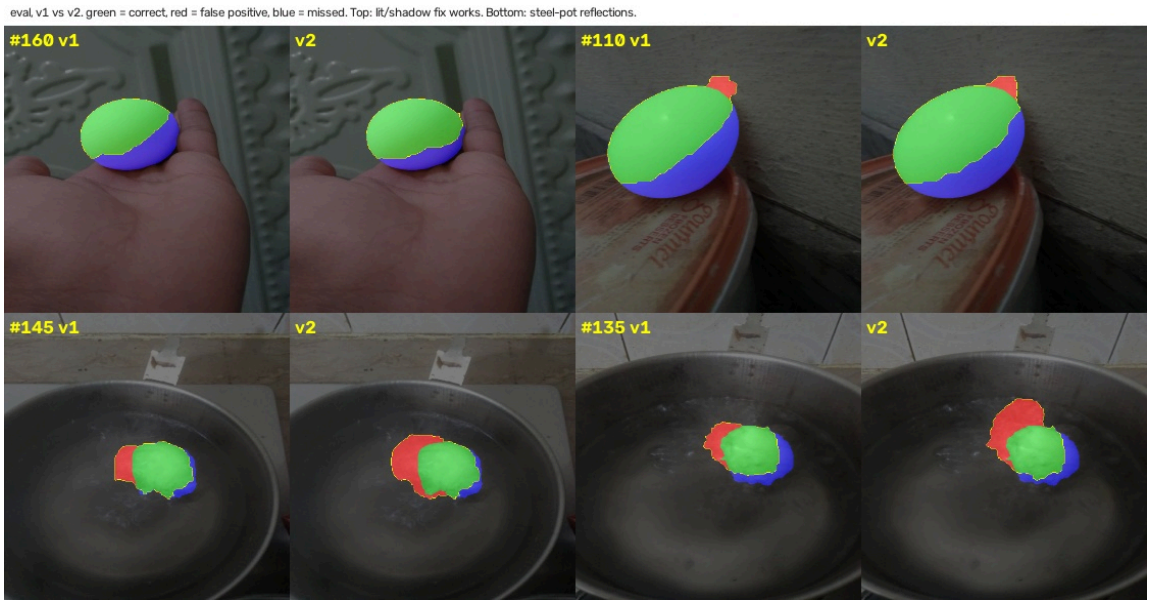
v2. Before GrabCut, an ellipse is fitted to the outline points that are strong edges *for that blob* (gradient ≥ 0.3 × the 90th percentile along its own outline) and added to the blob, with guards on growth, aspect, containment and colour.

- **Fixed Canny thresholds failed** (t17a): the whole outline is faint on pale backgrounds.
- **Dev: 0.786 → 0.809** mIoU, boundary F1 0.699 → 0.732.

Eval, run once: 0.786 vs v1's 0.789.

- **It helps the 3 eval scenes with lit/shadow splits:** green door 0.82 → 0.89, cloth 0.63 → 0.65, tub 0.77 → 0.78.
- **It hurts the steel pot** (0.86 → 0.83): the ellipse extends onto colourless reflections and bubbles in the water next to the egg.

Dev had no such scene. **v1 remains the submitted model.** Tuning v2 on the steel-pot images would mean designing on eval. This is also why the scene-grouped split mattered: with a random split, steel-pot images would have been in dev, the problem would have been tuned away unnoticed, and eval would have flattered the method.



Task 4: Video tracking

Video: "Raw and Boiled Egg Spinning", 60 s, 24 fps. The 10–42 s part shows two white eggs on a glossy table that are spun, **picked up, carried and put down** by a hand; the rest is title cards. Output video: **(Google Drive link to be added)**.

4.1 Detection

The v1 algorithm runs on every frame, in parallel across CPU cores, at 640 px wide. Its pixel parameters were tuned at that scale, and the output is drawn at the original resolution (the run used a 1080p copy). Two settings change for video (`configs/video_v1.yaml`):

- **Smaller size limits:** eggs cover about 0.5% of a frame, against 2.6–38% in the dataset.
- **No forced egg,** so the title cards stay empty.

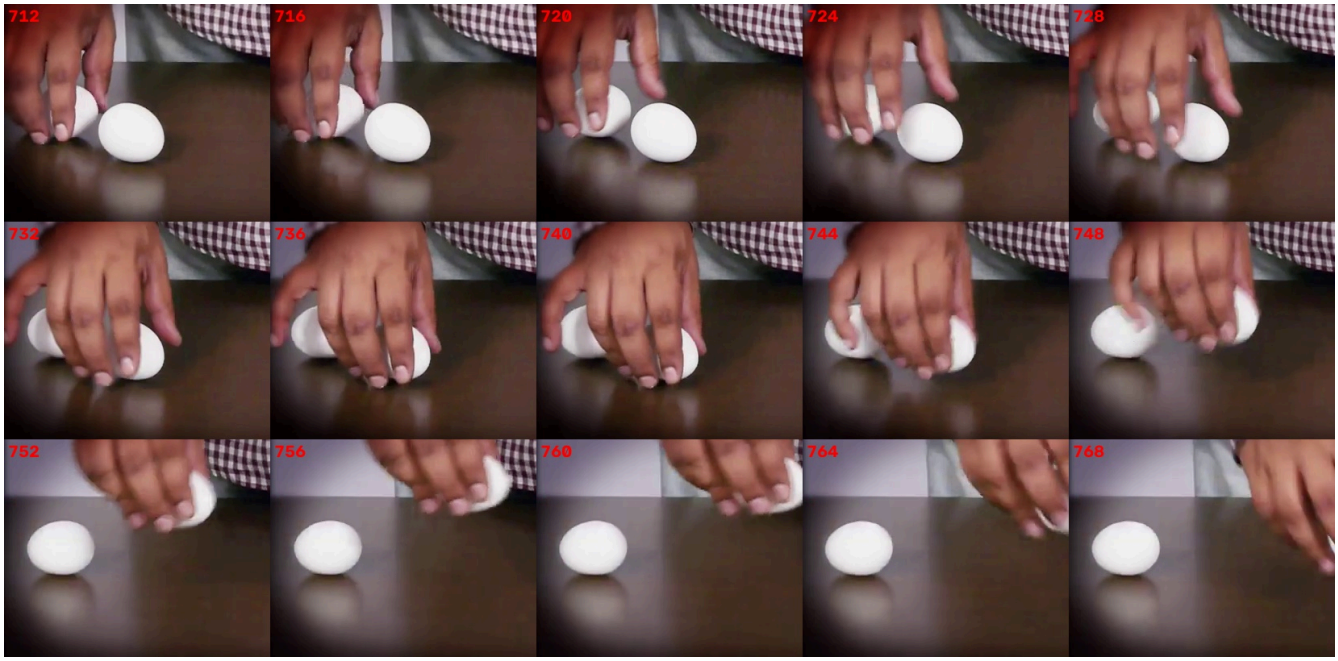
On sampled frames it detected both eggs and nothing on the text overlays or the logo. The only false detections were an occasional tiny speck or an egg merged with the table below it into one blob; the tracker filters those (no new track from a duplicate or from the reflection zone under an egg). While an egg is held, its outline is broken up by fingers and it is **not detected**.

4.2 Tracker (`src/track_video.py`)

- **Visible egg:**
  - A Kalman filter (x, y, vx, vy) predicts the next position.
  - Detections are matched with **Hungarian assignment** (independent of detection order) on centroid distance minus box IoU.
  - Matches are limited by the egg's typical size (median of recent detections), its vertical movement (eggs slide sideways in the image; a jump straight down is the reflection) and its area.
  - New tracks are confirmed after 3 hits.
- **Hidden or held egg: the track is kept.** Each frame, its estimate is moved by the **optical flow** (pyramidal Lucas–Kanade, forward-backward checked) of whatever covers it. Under a hand that is the hand's motion, so the estimate is carried along. The estimate then locks onto the **visible white part of the held egg** (a compact egg-white blob near the estimate; blobs are shared out one-to-one between hidden eggs).
- **Recovery:** a reappearing egg is matched to the **propagated** estimate, only if it's a whole egg, and never if it's another egg or a reflection. New IDs are only issued when no hidden egg can explain the detection.
- **Dropping tracks:** only at a scene change, when an egg leaves the frame, or after 30 s hidden.
- **Logging:** every new ID, miss and recovery (with distance and confidence) goes to `tracking_log.csv`.
- **Display:** a hidden egg keeps its "Egg N" label, on a dashed box at its estimated position.

4.3 The ID switch at 30 s: root cause and fix

What happens in the video. Frame-by-frame viewing showed that at 30 s one hand holds egg A at the far left and grabs egg B with its outer fingers. It leaves A on the table and carries B to the right.

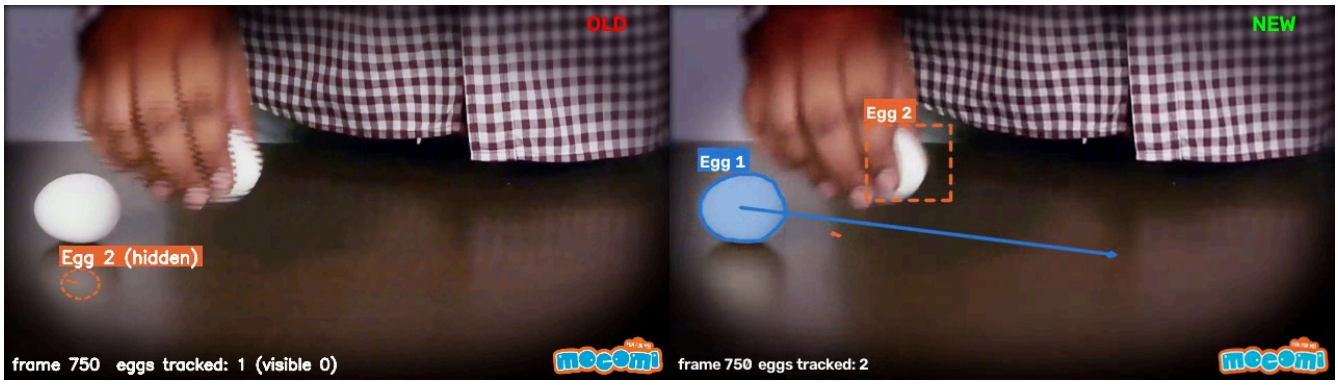


Frames 712–768: the hand holds A (left), grabs B, leaves A at the far left and carries B away.

Root causes in the first tracker:

1. **Held eggs are never detected.**
2. **A hidden egg's estimate stayed where it was last seen,** with a damped Kalman prediction. A was carried across the frame while its estimate sat on the right.
3. **Reappearing eggs were given the ID of the nearest *stale* position,** greedily. A reappeared at the far left, nearest to B's old spot, and B reappeared near A's old spot. **The IDs swapped.**
4. **Tracks were dropped after 2 s hidden.**
5. **Hidden eggs could be recovered onto the reflection** under an egg.

The fix is the hidden-egg handling above: estimates follow the hand, recovery uses the propagated estimates, and reflections can't recover a track.



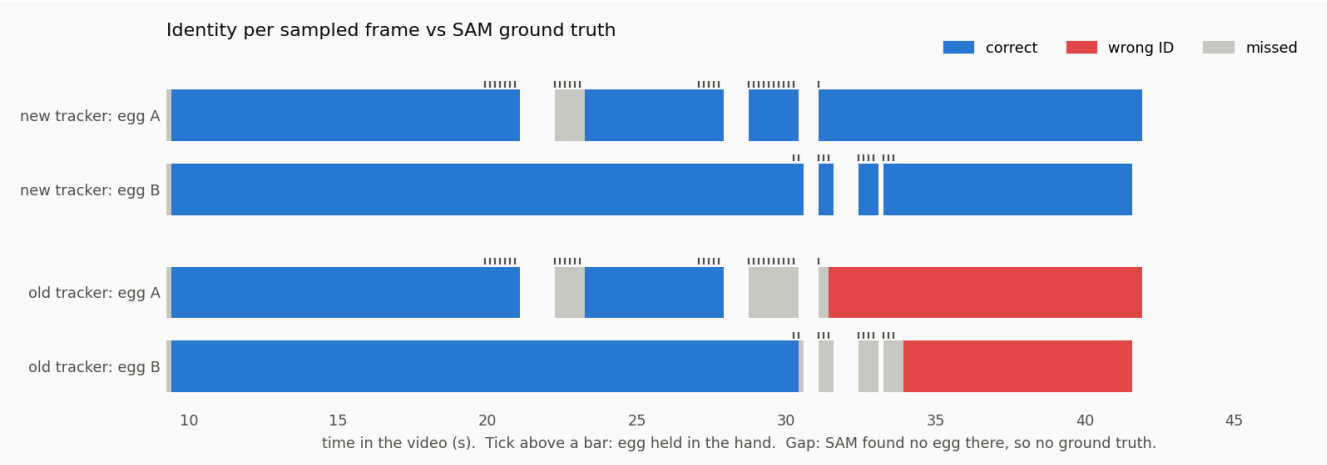
Frame 750, old (left) vs new (right). Old: "Egg 2" is on the wrong egg and the hidden label is stranded. New: each ID stays on its own egg, and Egg 2 is followed in the hand.

4.4 Quantitative evaluation

Ground truth. Grounding DINO + SAM (as in Task 1) on **every 4th frame** of the egg scene (196 frames). It is independent of the classical detector. Identities were assigned from the frame-by-frame account and checked on review sheets, giving 365 egg instances, 41 of them held.

Metrics (`src/eval_tracking.py`): centroid matching normalized by the ground-truth box diagonal (< 0.5), then MOTA, IDF1 and ID switches. False positives are only counted in frames where SAM found both eggs.

Tracker	MOTA	IDF1	ID switches	frames with wrong ID	held eggs followed	held-egg IDF1
final (`tracked_eggs.mp4`)	**0.962**	**0.953**	**0**	**0**	**85%**	**0.921**
first version	0.882	0.620	2	109 (from frame 754)	32%	0.481



The remaining 8 misses are:

- **Frame 222:** the first frame of the scene, before the 3-hit confirmation.
- **Frames 534–554:** an egg held low in the hand, where the estimate sits about half an egg above it. The ID is still correct.

Limitations:

- **The ground truth is SAM plus my own identity labels.**
- **Every 4th frame is sampled.** Per-frame checkpoints (`src/check_tracks.py`) cover the full frame rate.
- **The eggs are identical,** so identity rests on motion continuity. A fully hidden egg moving without anything visible would only be a best guess; the tracker flags such recoveries as `low_confidence`, and none occurred.

Reproducibility and use of AI tools

Reproducing everything

```
pip install -r requirements.txt
python src/prepare_data.py                                  # dataset -> data/images
# Task 1 (GPU): notebooks/task1_sam_kaggle.ipynb on Kaggle -> outputs/sam_masks (committed)
python src/make_split.py                                    # Task 2 split
python src/run_experiment.py configs/t16_nest_cap4.yaml     # any trial on dev
python src/run_experiment.py configs/t16_nest_cap4.yaml --split eval --save-masks
python src/run_experiment.py configs/t16_nest_cap4.yaml --save-masks --no-log
python src/evaluate.py && python src/stats.py && python src/ablation.py   # Task 3
python src/track_video.py --video data/video/egg_video.mp4  # Task 4 (or
notebooks/task4_tracking_kaggle.ipynb)
python src/eval_tracking.py outputs/video/tracks.csv        # needs the video GT
(notebooks/task4_video_gt_kaggle.ipynb)
python src/build_report.py --drive-link <link>              # this report
```

Use of AI tools

This project was developed with **Claude Code** (Anthropic's AI coding assistant), used interactively throughout.

- **What Claude Code did:**
  - wrote most of the code: notebooks, segmentation pipeline, metrics, tracker and analysis scripts;
  - ran the experiments and produced the diagnostics and figures;
  - drafted this report.
- **What I did:**
  - set the direction and decided what to pursue at each step (for example the Kaggle/GPU setup, going beyond the baseline, the root-cause investigation of the tracking ID switch, and the robustness work);
  - ran the GPU notebooks;
  - reviewed the results.
- **How the results were checked:** every number in this report comes from the scripts in the repository and can be regenerated with the commands above, and each stated finding was checked against the result files and figures before being written down.